Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



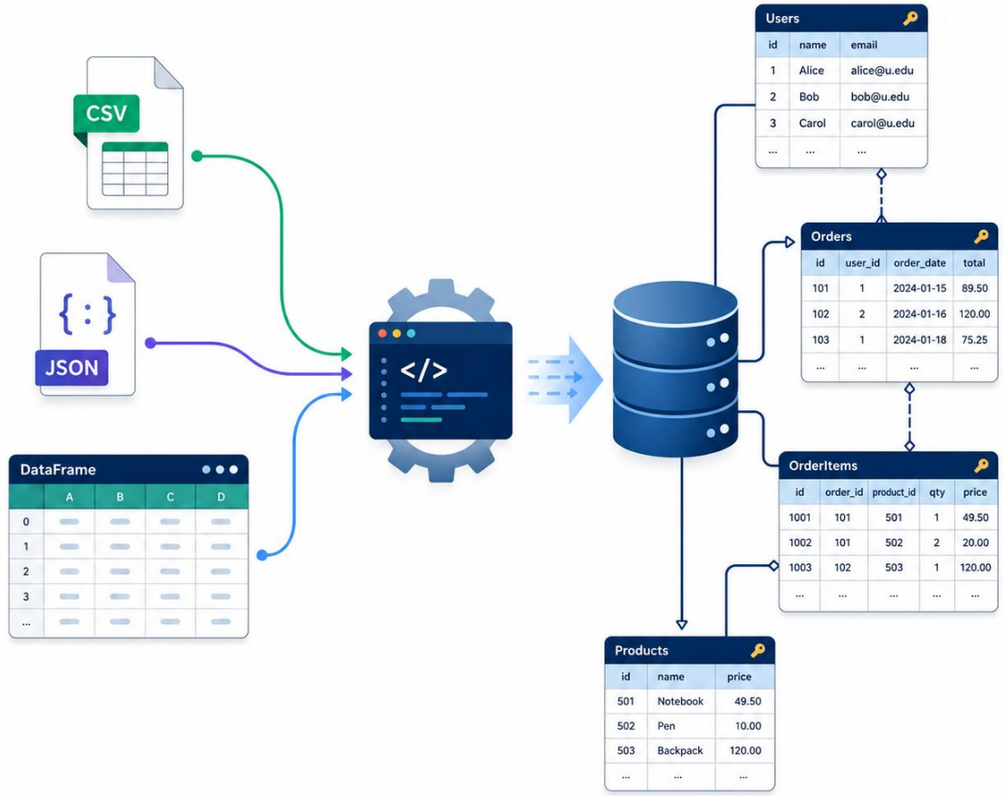
IIC2115 - Programación como Herramienta para la Ingeniería

Almacenamiento y consulta de datos

Profesores: Camila Vera / Hans Löbel

¿Por qué no basta con guardar todo en archivos?

- Al principio, un archivo CSV o JSON, o incluso un DataFrame pueden ser suficientes.
- Pero los problemas aparecen cuando los datos crecen, se relacionan o deben actualizarse.
- Una base de datos permite organizar datos relacionados, evitar inconsistencias y consultar información de forma eficiente.



El problema de una sola tabla gigante

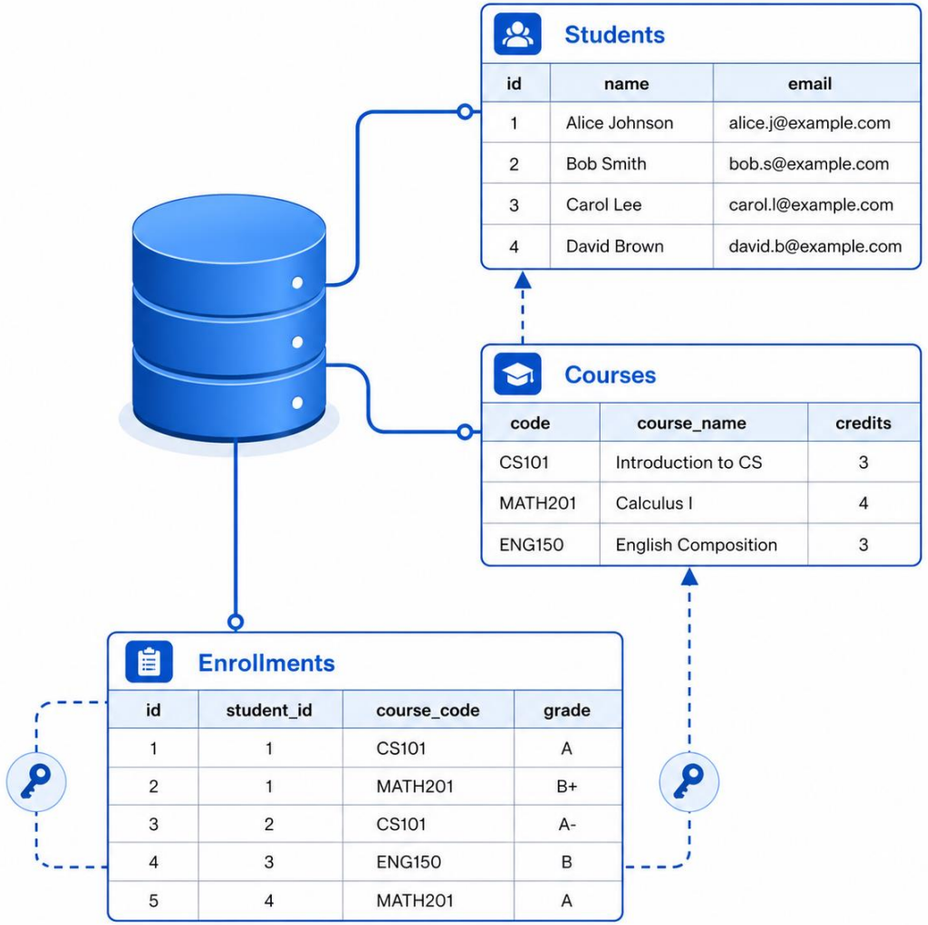
- Una tabla única puede parecer simple.
- Pero suele repetir información.
- También mezcla entidades distintas.
- Y hace difícil mantener consistencia.

student_name	student_email	course_code	course_name	professor	semester	grade
María González	maria.gonzalez@uc.cl	IIC2115	Bases de Datos	Ana Pérez	2024-1	6.5
María González	maria.gonzalez@uc.cl	MAT1203	Álgebra Lineal	Luis Ramirez	2024-1	6.0
María González	maria.gonzalez@uc.cl	IIC1103	Programación I	Jorge Castillo	2024-1	5.5
Juan Pablo Silva	juan.silva@uc.cl	IIC2115	Bases de Datos	Ana Pérez	2024-1	6.0
Juan Pablo Silva	juan.silva@uc.cl	MAT1203	Álgebra Lineal	Luis Ramirez	2024-1	5.5
Camila Torres	camila.torres@uc.cl	IIC2115	Bases de Datos	A. Perez ⚠	2024-1	5.0
Camila Torres	camila.torres@uc.cl	IIC1103	Programación I	Jorge Castillo	2024-1	6.0
Diego Morales	diego.morales@uc.cl	MAT1203	Algebra Lineal	Luis Ramirez	2024-1	5.5

repetido repetido repetido ⚠ inconsistencia

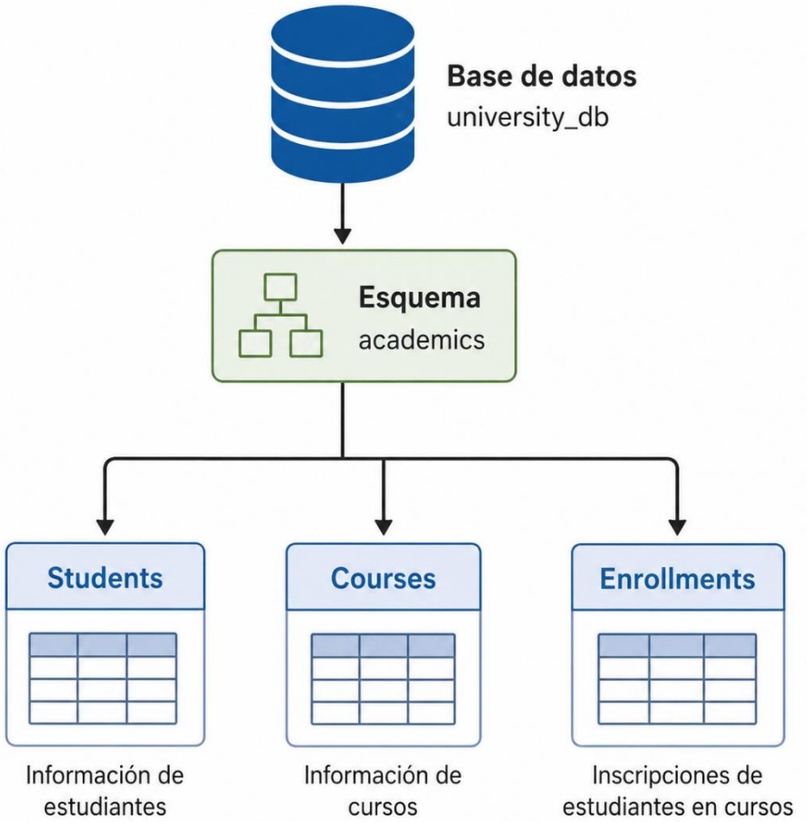
¿Qué es una base de datos?

- Es un conjunto de datos de un mismo contexto, almacenados bajo una lógica común.
- Permite guardar, consultar, actualizar y relacionar información.
- En una base de datos relacional, los datos se organizan en tablas conectadas entre sí.



¿Qué es una base de datos?

- Una base de datos puede contener varias tablas.
- Un esquema describe cómo se organizan esas tablas.
- Las tablas representan entidades o relaciones relevantes del problema.
- Una base de datos también podría contener múltiples esquemas.



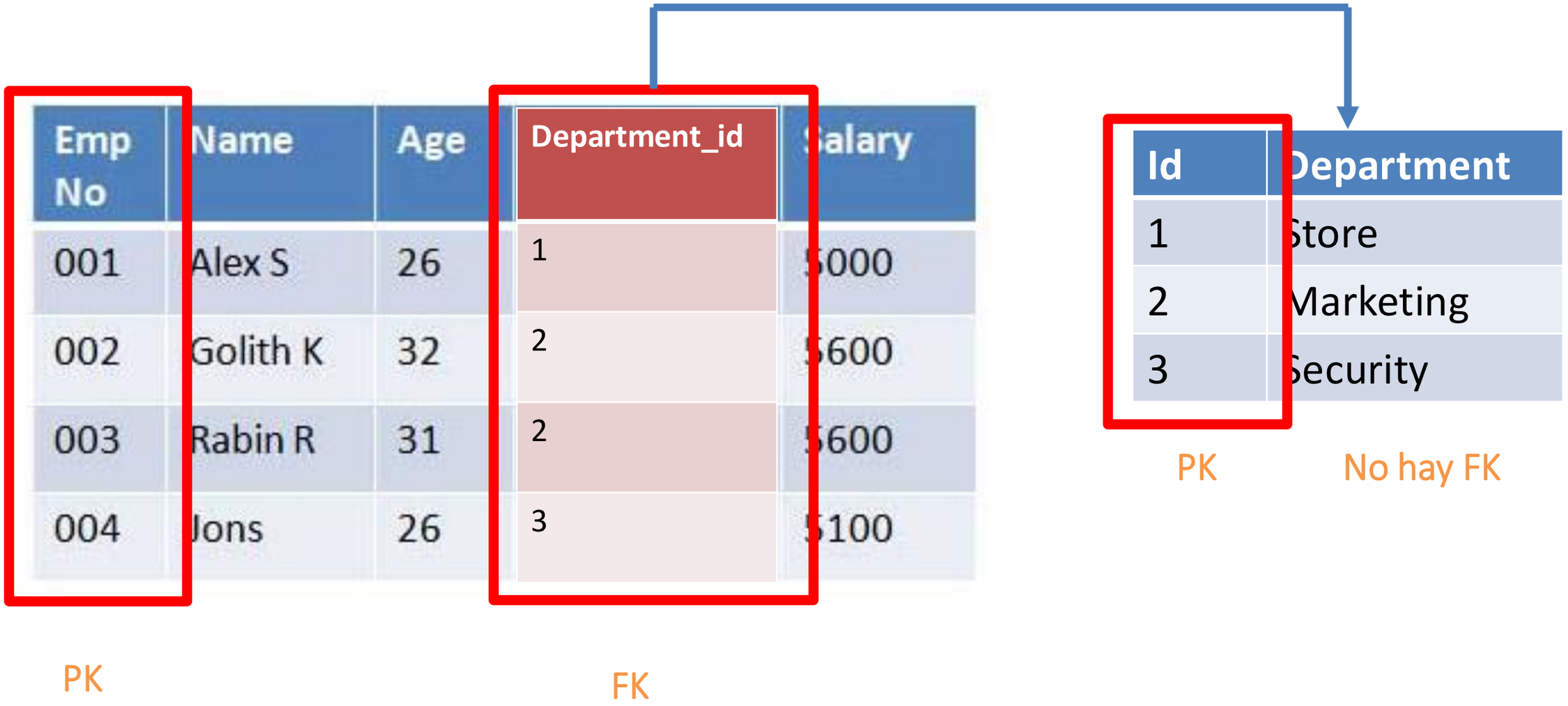
¿Qué contiene una tabla?

Columna: representa un atributo de las entidades.
Cada columna tiene un solo tipo de dato.

Fila: corresponde a un registro o instancia

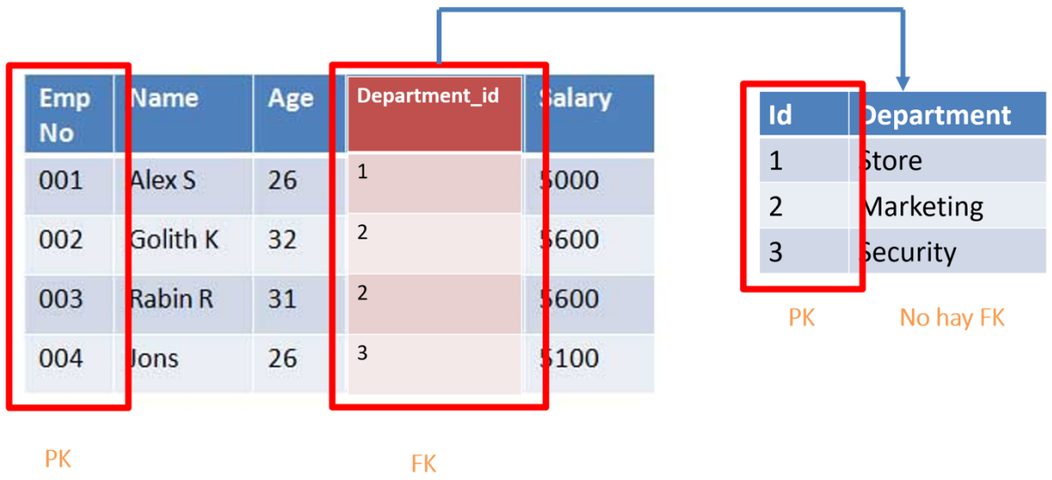
Emp No	Name	Age	Department	Salary
001	Alex S	26	Store	5000
002	Golith K	32	Marketing	5600
003	Rabin R	31	Marketing	5600
004	Jons	26	Security	5100

Llaves primarias y secundarias (primary keys y foreign keys)



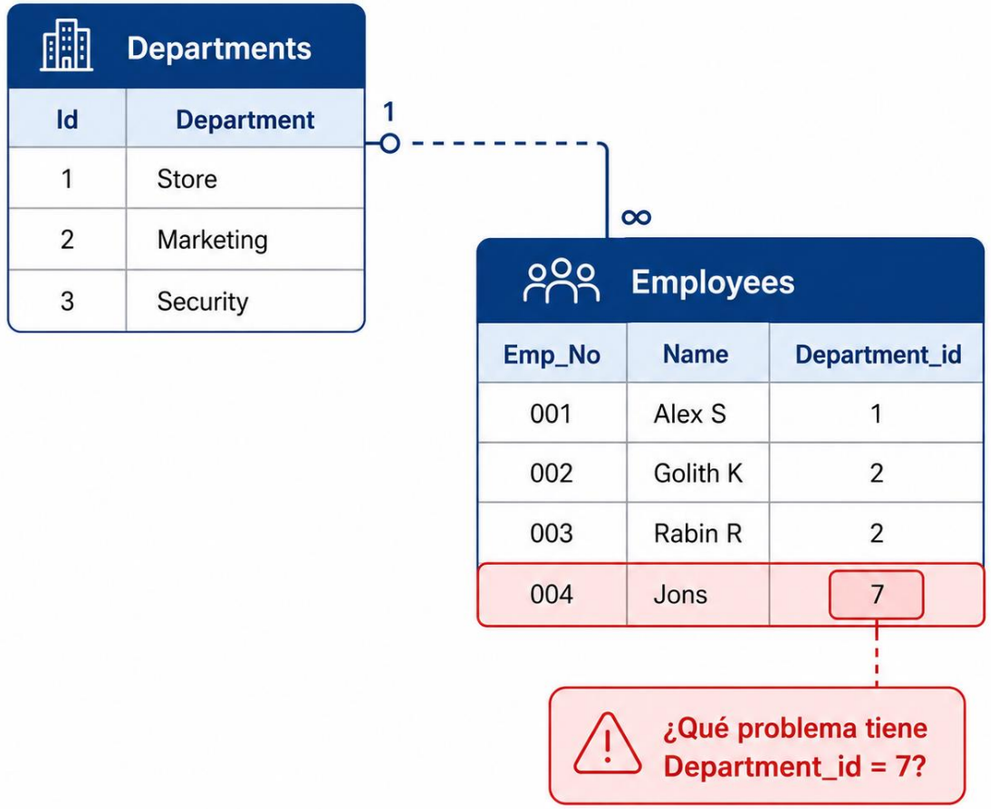
Llaves primarias y secundarias (primary keys y foreign keys)

- Una llave primaria identifica de forma única cada fila de una tabla.
- Una llave foránea conecta una tabla con otra.
- Estas llaves permiten representar relaciones entre entidades.
- Sin llaves, las relaciones quedan implícitas y son más difíciles de controlar.



Restricciones de integridad: reglas para mantener datos consistentes

- Una base de datos no solo guarda datos, también puede imponer reglas.
- (casi) Todas las tablas deben definir un identificador único: PRIMARY KEY.
- Una relación entre tablas debe apuntar a datos existentes: FOREIGN KEY.
- Algunas columnas no deberían aceptar valores vacíos o repetidos: NOT NULL, UNIQUE.



Structured Query Language (SQL) → El lenguaje para trabajar con bases relacionales

- Lenguaje de definición de datos (DDL)
  - Creación
  - Inserción
  - Eliminación
  - Modificación de definiciones de tablas.

  *Las restricciones de integridad se pueden definir en tablas, ya sea cuando se crea la tabla o posteriormente.

- Lenguaje de manipulación de datos (DML)
  - Consultas

- En este curso utilizaremos SQLite, una versión reducida de SQL, que tiene algunas diferencias menores en sus sintaxis.



Creación

CREATE TABLE [IF NOT EXISTS] table_name (column_1 data_type, column_2 data_type, ...)

Emp No	Name	Age	Department	Salary
001	Alex S	26	Store	5000
002	Golith K	32	Marketing	5600
003	Rabin R	31	Marketing	5600
004	Jons	26	Security	5100

CREATE TABLE Employees (EmpNo TEXT, Name TEXT, Age INTEGER, Department TEXT, Salary INTEGER)

Inserción

INSERT INTO table_name (column1,column2 ,..) **VALUES**(value1, value2 ,...)

Emp No	Name	Age	Department	Salary
001	Alex S	26	Store	5000
002	Golith K	32	Marketing	5600
003	Rabin R	31	Marketing	5600

INSERT INTO Employees (EmpNo, Name, Age**,** Department, Salary) **VALUES** (‘004’, ‘Jons’, 26, ‘Security’, 5100)

Actualización

UPDATE table_name **SET** column_1 = new_value_1, column_2 = new_value_2
WHERE search_condition

Emp No	Name	Age	Department	Salary
001	Alex S	26	Store	5000
002	Golith K	32	Marketing	5600
003	Rabin R	31	Marketing	5600
004	Jons	26	Marketing	5100

UPDATE Employees E **SET** E.Department = ‘Marketing’ **WHERE** E.EmpNo = ‘004’

Eliminación

DELETE FROM table_name WHERE search_condition;

Emp No	Name	Age	Department	Salary
001	Alex S	26	Store	5000
002	Golith K	32	Marketing	5600
003	Rabin R	31	Marketing	5600

DELETE FROM Employees E WHERE E.EmpNo = ‘004’

Creación de tablas conectadas con *Primary Key* y *Foreign Key*



Emp No	Name	Age	Department_id	Salary
001	Alex S	26	1	5000
002	Golith K	32	2	5600
003	Rabin R	31	2	5600
004	Jons	26	3	5100

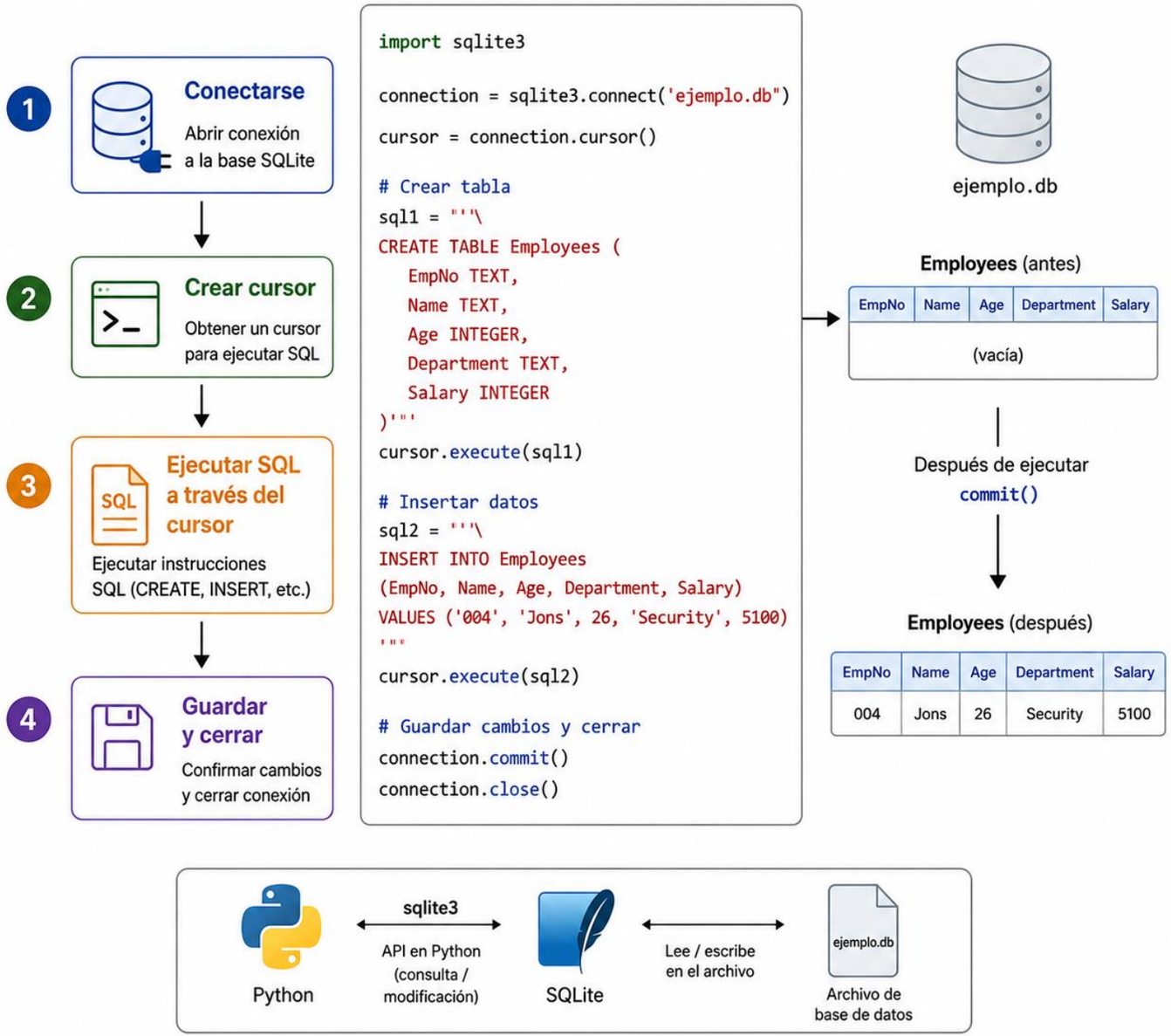
Id	Department
1	Store
2	Marketing
3	Security

CREATE TABLE Departments (Id **INTEGER** **PRIMARY KEY**, Department **TEXT**)

CREATE TABLE Employees (EmpNo **TEXT** **PRIMARY KEY**, Name **TEXT**, Age **INTEGER**, Department_id **INTEGER**, Salary **INTEGER**, **FOREING KEY** (Department_id) REFERENCES Departments)

Uso desde Python: crear y cargar datos

- Python puede conectarse a una base SQLite usando sqlite3.
- Un cursor permite ejecutar instrucciones SQL.
- **commit()** guarda los cambios realizados.
- **close()** cierra la conexión.



Consultas: SELECT, FROM, WHERE

```
SELECT
        column_list
FROM
        table_list
WHERE
        row_filter
```

Name	Age
Golith K	32
Rabin R	31

```
SELECT * FROM Emnployees

SELECT * FROM Employees E WHERE E.Age > 30

SELECT Name, Age FROM Employees E WHERE E.Age > 30
```

Joins: combinar tablas para responder mejores preguntas

```
SELECT
        column_list
FROM
        table_list
WHERE
        row_filter
```



Emp No	Name	Age	Department_id	Salary
001	Alex S	26	1	5000
002	Golith K	32	2	5600
003	Rabin R	31	2	5600
004	Jons	26	3	5100

Id	Department
1	Store
2	Marketing
3	Security

SELECT Name **FROM** Employees E **JOIN** Departments D **ON** E.Department_id = D.id **WHERE** D.Department = ‘Store’

ALEX S

Joins: combinar tablas para responder mejores preguntas

```
SELECT
        column_list
FROM
        table_list
WHERE
        row_filter
```



Emp No	Name	Age	Department_id	Salary
001	Alex S	26	1	5000
002	Golith K	32	2	5600
003	Rabin R	31	2	5600
004	Jons	26	3	5100

Id	Department
1	Store
2	Marketing
3	Security

SELECT Name **FROM** Employees E, Departments D **WHERE** E.Department_id = D.id **AND** D.Department = ‘Store’

ALEX S

Anidación

Emp No	Name	Age	Department	Salary
001	Alex S	26	Store	5000
002	Golith K	32	Marketing	5600
003	Rabin R	31	Marketing	5600
004	Jons	26	Security	5100

```
SELECT Name FROM (SELECT Name, Salary FROM Employees E WHERE E.Age < 30)
WHERE Salary >= 5100
```

JONS

Otras funciones importantes

- ¿Cuáles son los registros más recientes o más antiguos? -> ORDER BY
- ¿Cuántos registros hay por categoría? -> GROUP BY, COUNT
- ¿Qué grupos cumplen cierta condición? -> HAVING
- ¿Cuál es el promedio, máximo o mínimo de una variable? -> AVG, MAX, MIN
- ¿Qué registros pertenecen a cierto conjunto? -> IN

Uso en Python: ejecutar consultas

- Desde Python podemos ejecutar consultas SQL.
- **fetchone()** recupera una fila.
- **fetchall()** recupera todas las filas restantes.
- El resultado puede procesarse luego con Python.

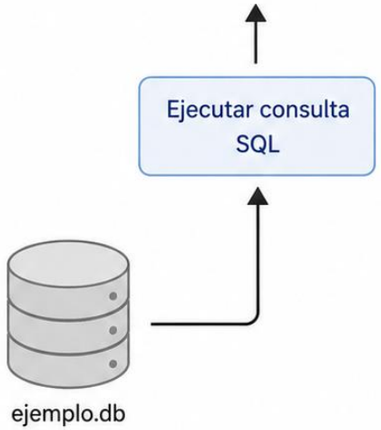
```python
import sqlite3

connection = sqlite3.connect("ejemplo.db")
cursor = connection.cursor()

cursor.execute("SELECT * FROM Employees")

one_row = cursor.fetchone()
all_rows = cursor.fetchall()

connection.close()
```



Resultados

one_row = cursor.fetchone()

(una fila)

```
('001', 'Alex S', 32, 'Store', 4500)
```

all_rows = cursor.fetchall()

(todas las filas restantes)

```
[
 ('002', 'Golth K', 28, 'Marketing', 4800),
 ('003', 'Rabin R', 35, 'Marketing', 5200),
 ('004', 'Jons', 26, 'Security', 5100),
 ('005', 'Maya L', 40, 'Store', 5600)
]
```

Employees

Emp_No	Name	Age	Department	Salary
001	Alex S	32	Store	4500
002	Golith K	28	Marketing	4800
003	Rabin R	35	Marketing	5200
004	Jons	26	Security	5100
005	Maya L	40	Store	5600

Construir consultas con strings puede ser peligroso

```
edad = '50; DROP TABLE Empleado'

def mayores_que(edad):
    connection = sqlite3.connect('ejemplo.db')
    cursor = connection.cursor()

    sqlStatement = 'SELECT * FROM Empleados E WHERE E.Age > {}'.format(edad)

    cursor.execute(sqlStatement)
    resp = cursor.fetchall()
    connection.close()
    return resp
```

Uso en Python: parametrización sin usar strings (forma correcta)

```
edad = ‘50; DROP TABLE Empleado’

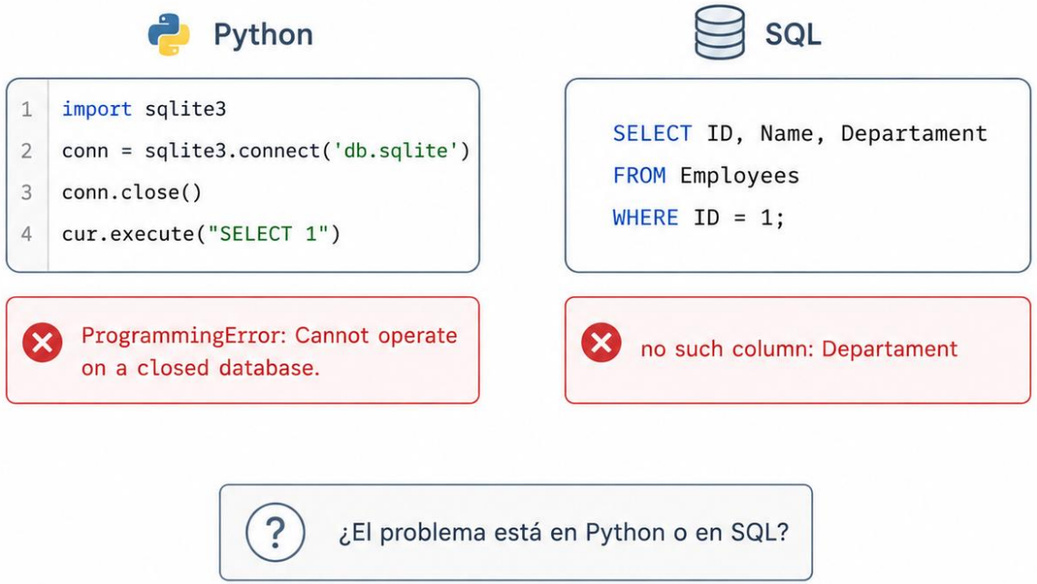
def mayores_que(edad):
    connection = sqlite3.connect(‘ejemplo.db’)
    cursor = connection.cursor()

    sqlStatement = ‘SELECT * FROM Empleados E WHERE E.Age > ?’

    cursor.execute(sqlStatement, (edad,))
    resp = cursor.fetchall()
    connection.close()
    return resp
```

Errores frecuentes al trabajar con SQL desde Python

- Errores de Python: conexión cerrada, variables mal definidas, tipos incorrectos.
- Errores SQL: tabla inexistente, columna mal escrita, sintaxis inválida.
- Resultados vacíos no siempre significan error: a veces el filtro no encuentra datos.
- Una buena estrategia es probar primero la consulta, p.ej., con DB Browser for SQLite, y luego integrarla en Python.



Cuáles son los aspectos centrales de esta primera parte del capítulo

- Una base relacional organiza datos en tablas conectadas.
- Las llaves permiten representar relaciones y mantener consistencia.
- SQL permite crear, modificar y consultar datos.
- Python permite automatizar el trabajo con bases de datos.
- En los notebooks está el detalle operativo para resolver los ejercicios.

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2115 - Programación como Herramienta para la Ingeniería

Almacenamiento y consulta de datos

Profesores: Camila Vera / Hans Löbel